

HCCL通信库算子开发介绍

—— AICPU+TS & CCU通信引擎

作者：李标智

时间：2026/06/16

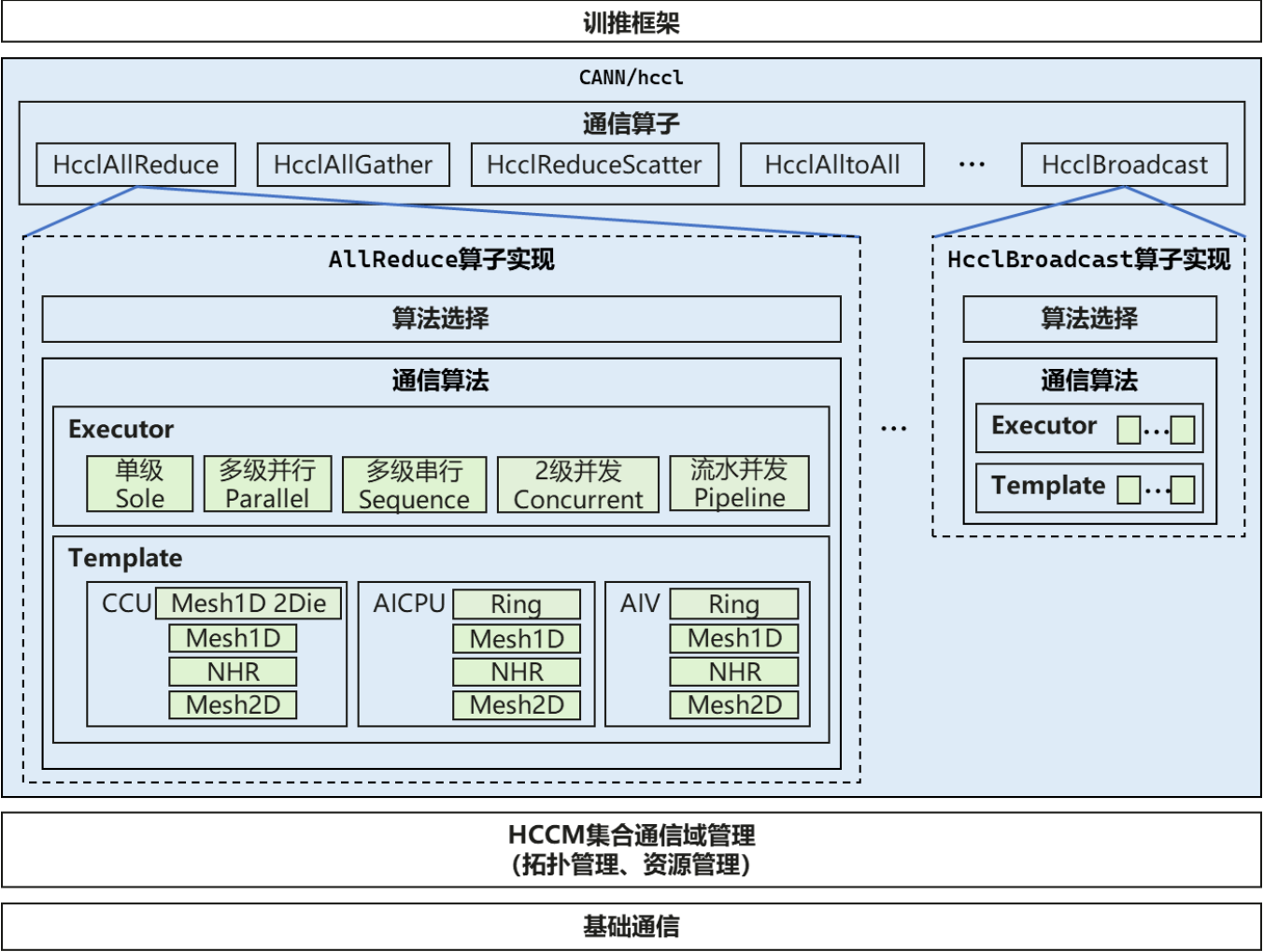
目录

Part 1 HCCL算子层软件架构

Part 2 HCCL算子开发-AICPU+TS引擎

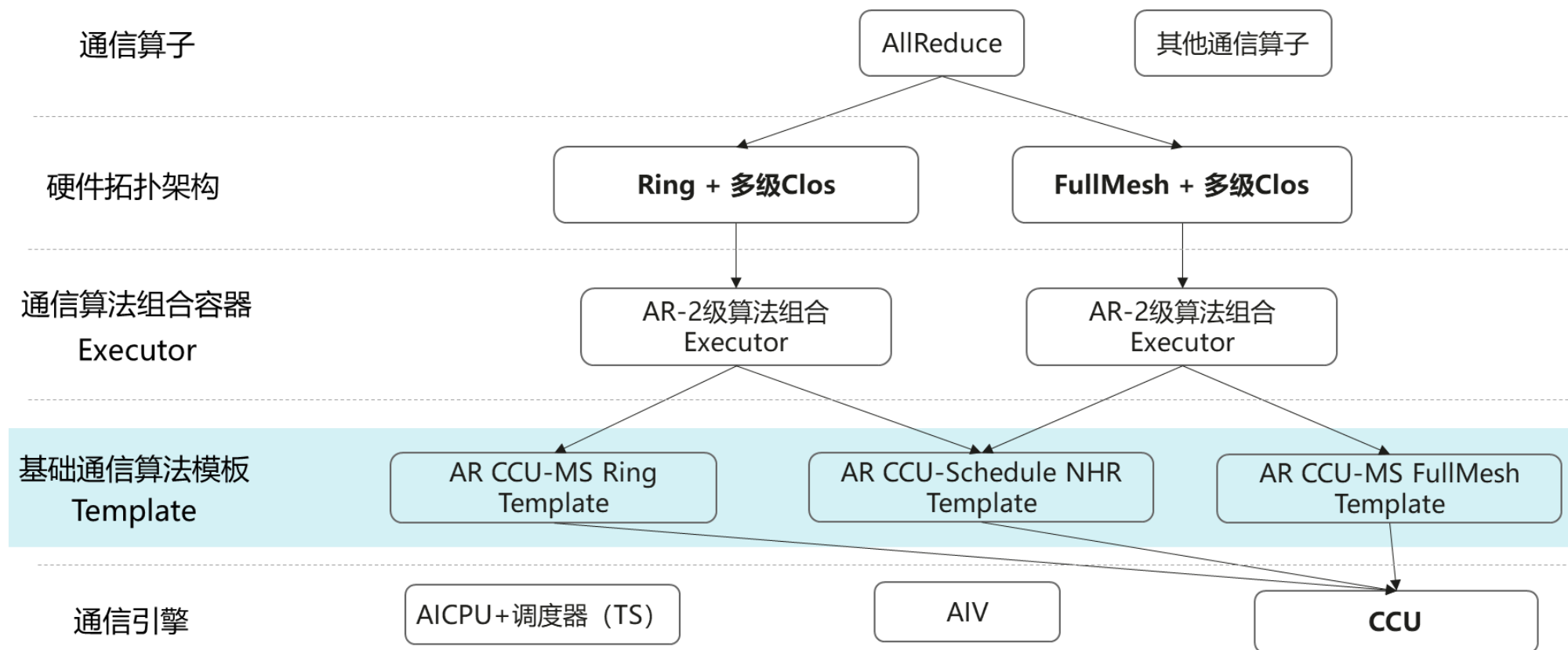
Part 3 HCCL算子开发-CCU引擎

HCCL算子开发最佳实践：CANN/hccl软件架构



- **通信算子:** 对上层AI框架提供统一的算子调用入口，内部完成集合通信算子的软件实现
- **算法 (Executor) 选择器:** 根据当前**集群拓扑**和**硬件形态**，选择最佳性能的集合通信算法，返回对应的Executor名称
- **Executor层:** 每一个Executor都提供了一个完整的算法实现，包括计算算法所需资源、计算通信关系。可以组合多个基础算法 (Template) 来实现分级算法功能，**灵活应对昇腾950及之后产品的多样化产品形态。**
- **Template层:** 实现算子实际任务编排逻辑，调用基础通信API，实现基础的集合通信算法

复杂拓扑下的架构实践：HCCL复用基础通信算法模板，通过Executor多维组合



- 1、基础通信算法模板数量受**通信算子**、**基础拓扑算法**、**通信引擎**三个因素影响
- 2、复用算法模板可降低开发工作量
- 3、第三方可**按需扩展**基础通信算法模板和Executor

目录

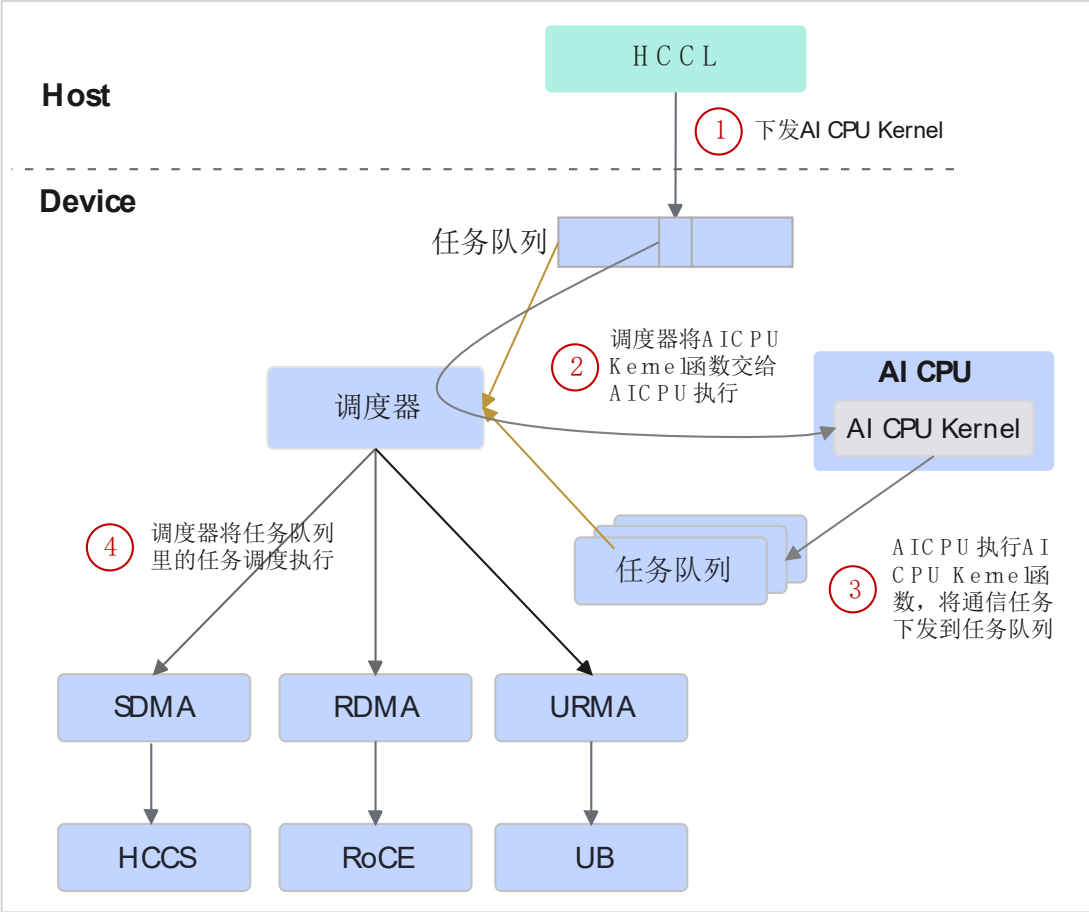
Part 1 HCCL算子层软件架构

Part 2 HCCL算子开发-AICPU+TS引擎

Part 3 HCCL算子开发-CCU引擎

HCCL算子执行流程：AICPU + TS通信引擎

AICPU+调度器(TS) 通信引擎算子执行流程



1. Host侧部分:

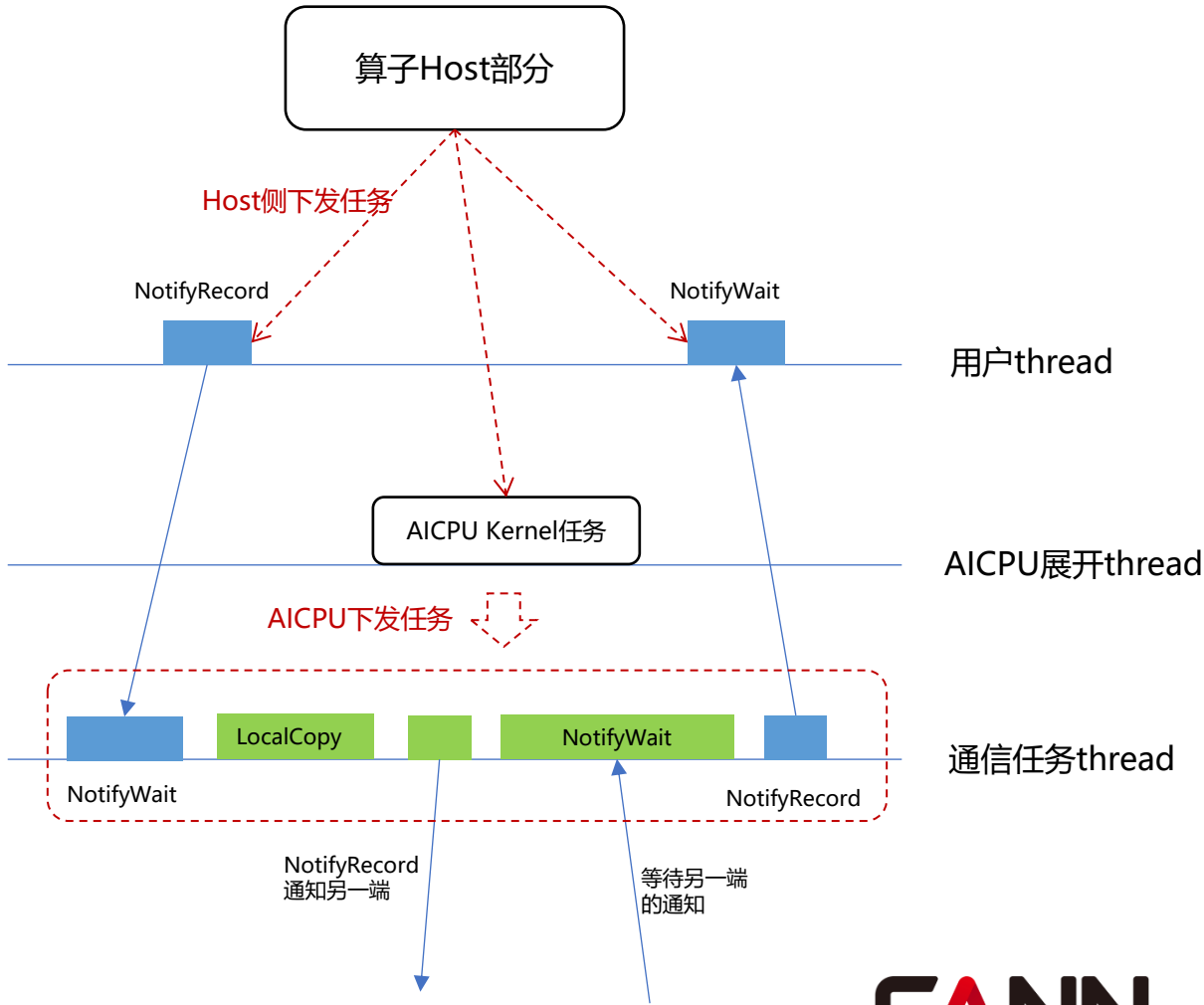
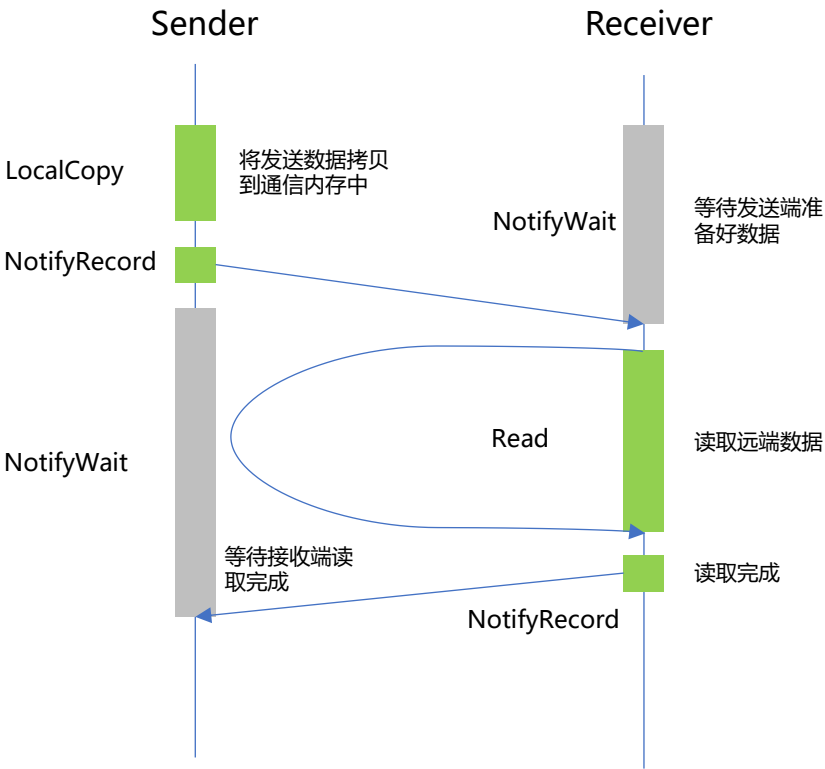
- 申请资源 (Channel、Thread等) ;
- 创建算子上下文缓存, 缓存资源句柄;
- 下发AICPU Kernel任务至Device任务队列。

2. Device侧流程:

- AICPU Kernel被任务调度器调度给AI CPU执行;
- AICPU执行算法逻辑, 将通信、同步等任务下发至其他任务队列;
- AI CPU提交的通信任务被调度器调度至执行器执行。

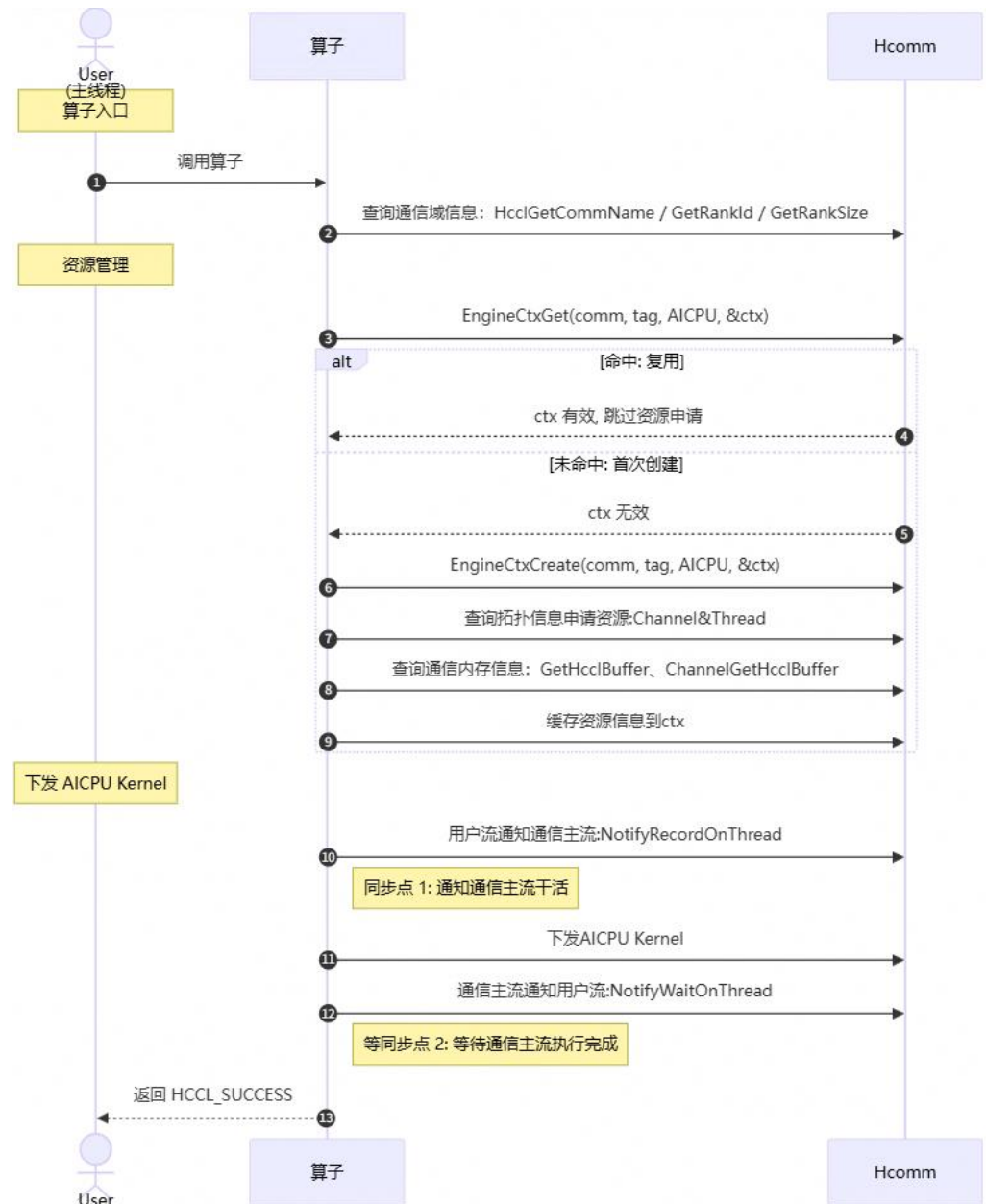
HCCL算子开发介绍：自定义点到点通信样例

- HcclSendCustom(sendBuf, count, dataType, destRank, comm, stream)
- HcclRecvCustom(recvBuf, count, dataType, srcRank, comm, stream)



HCCL算子开发介绍：自定义点到点通信样例

参考链接: https://gitcode.com/cann/hccl/blob/master/examples/04_custom_ops_p2p/op_host/send.cc



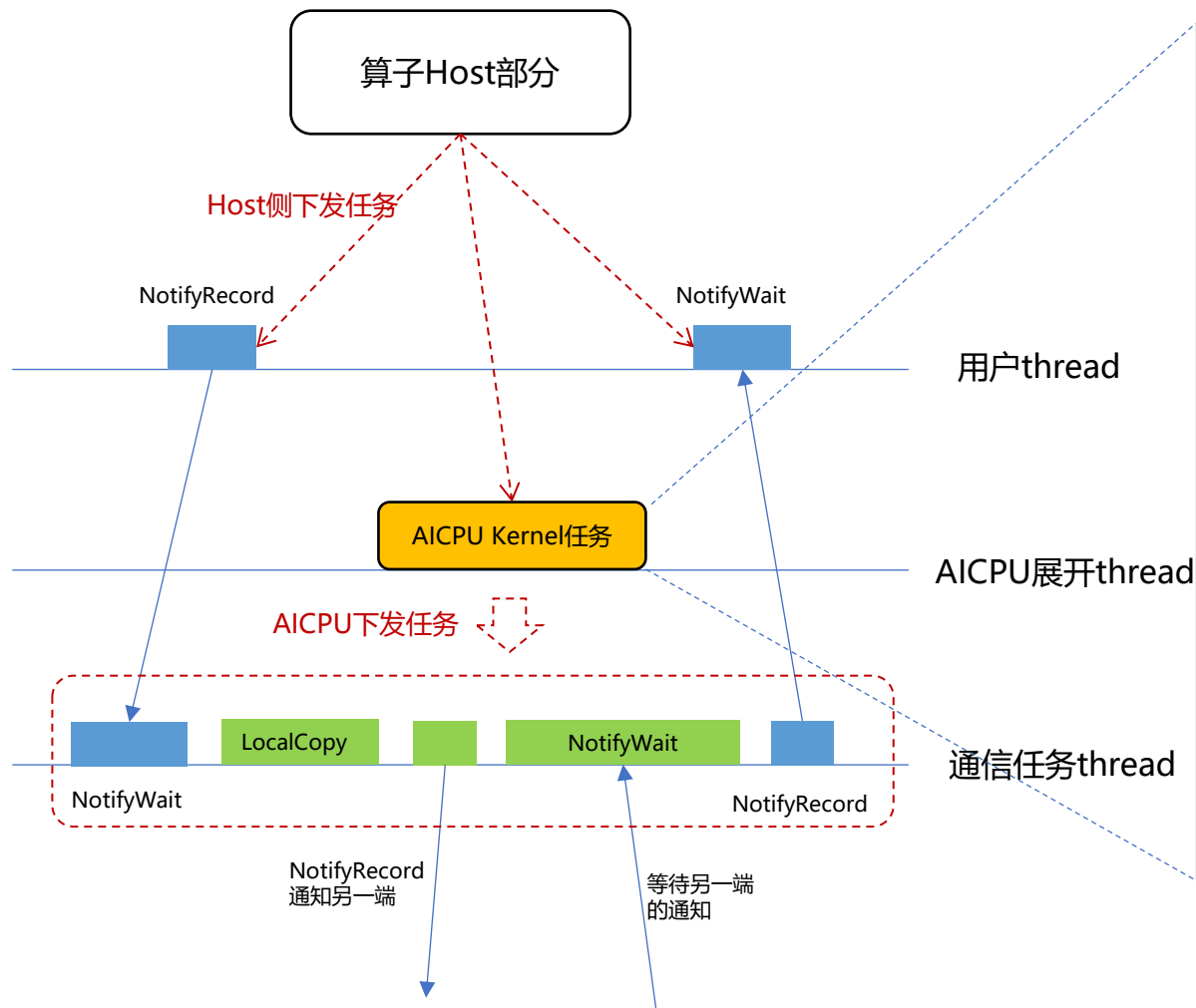
```
HcclResult HcclSendCustom(void *sendBuf, uint64_t count, HcclDataType dtype, uint32_t
destRank, HcclComm comm, aclrtStream stream)
{
    // 准备 OpParam
    OpParam param;
    // 查询通信域信息
    HcclGetRankId (comm, &rank);
    HcclGetRankSize (comm, &rankSize);
    // 资源管理: 按 tag 缓存 AlgResourceCtx
    void *ctx = nullptr;
    HcclResult rc = HcclEngineCtxGet(comm, param.tag, CommEngine::COMM_ENGINE_AICPU, &ctx,
    &size);
    if (rc != HCCL_SUCCESS) {
        HcclEngineCtxCreate(comm, param.tag, CommEngine::COMM_ENGINE_AICPU, size, &ctx);
        // 申请 Thread
        HcclThreadAcquireWithStream(comm, COMM_ENGINE_CPU_TS, stream, 1 /*notifyNum*/,
        &cpuThread);
        HcclThreadAcquire(comm, COMM_ENGINE_AICPU_TS, 1 /*threadNum*/, 1 /*notifyNum*/,
        &aicpuThread);
        // 建立 Channel
        HcclChannelDesc desc;
        desc.remoteRank = destRank;
        ...
        HcclChannelAcquire(comm, CommEngine::COMM_ENGINE_AICPU, &desc, 1, &channelHandle);
        // 查询通信 Buffer
        HcclGetHcclBuffer (comm, &localBuffer.addr, &localBuffer.size);
        HcclChannelGetHcclBuffer (comm, channelHandle, &remoteBuffer.addr, &remoteBuffer.size);
        // ctx 拷到 device 侧
        aclrtMemcpy(ctx, size, &resCtxHost, size, ACL_MEMCPY_HOST_TO_DEVICE);
    }
    param.resCtx = (AlgResourceCtx *)ctx;
    // Host stream 通知 AICPU 主 thread 开工

    HcclAicpuKernelLaunch(comm, func, args, ..., stream);
    return HCCL_SUCCESS;
}
```



HCCL算子开发介绍：自定义点到点通信样例

参考链接: https://gitcode.com/cann/hccl/blob/master/examples/04_custom_ops_p2p/op_kernel_aicpu/exec_op.cc



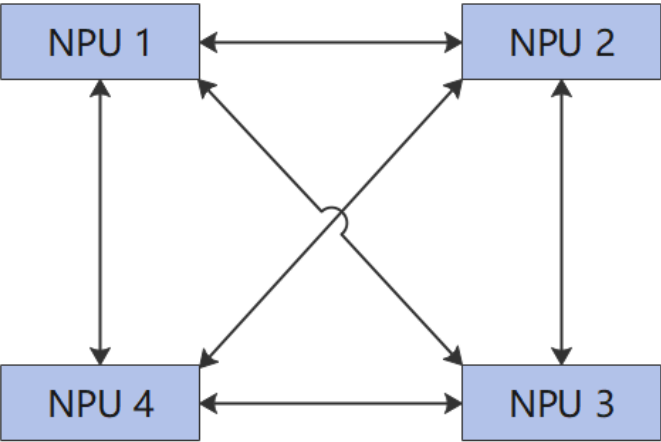
```
int HcclP2PAicpuKernel(OpParam *param) {  
    // 准备: 占 comm + 批模式 + 等 用户流同步  
    HcommAcquireComm(param->commName);  
    HcommBatchModeStart(param->tag);  
    HcommThreadNotifyWaitOnThread(param->resCtx->aicpuThread, 0, timeout);  
    // -----  
    // 算法编排  
    // -----  
    uint64_t size = param->count * SIZE_TABLE[param->dataType];  
    if (param->opType == HcclCMDType::HCCL_CMD_SEND) {  
        HcommLocalCopyOnThread (aicpuThread, localBuffer.addr, inputPtr, size);  
        HcommChannelNotifyRecordOnThread(aicpuThread, channel, NOTIFY_IDX_ACK);  
        HcommChannelNotifyWaitOnThread (aicpuThread, channel,  
        NOTIFY_IDX_DATA_SIGNAL, 1836);  
    } else { // RECEIVE  
        HcommChannelNotifyWaitOnThread (aicpuThread, channel, NOTIFY_IDX_ACK, 1836);  
        HcommReadOnThread(aicpuThread, channel, outputPtr, remoteBuffer.addr, size);  
        HcommChannelNotifyRecordOnThread(aicpuThread, channel,  
        NOTIFY_IDX_DATA_SIGNAL);  
    }  
    // 通知用户流  
    HcommThreadNotifyRecordOnThread(aicpuThread, cpuThreadOnAicpu, 0);  
    HcommBatchModeEnd(param->tag);  
    HcommReleaseComm(param->commName);  
    return 0;  
}
```

<https://gitcode.com/cann>

CANN

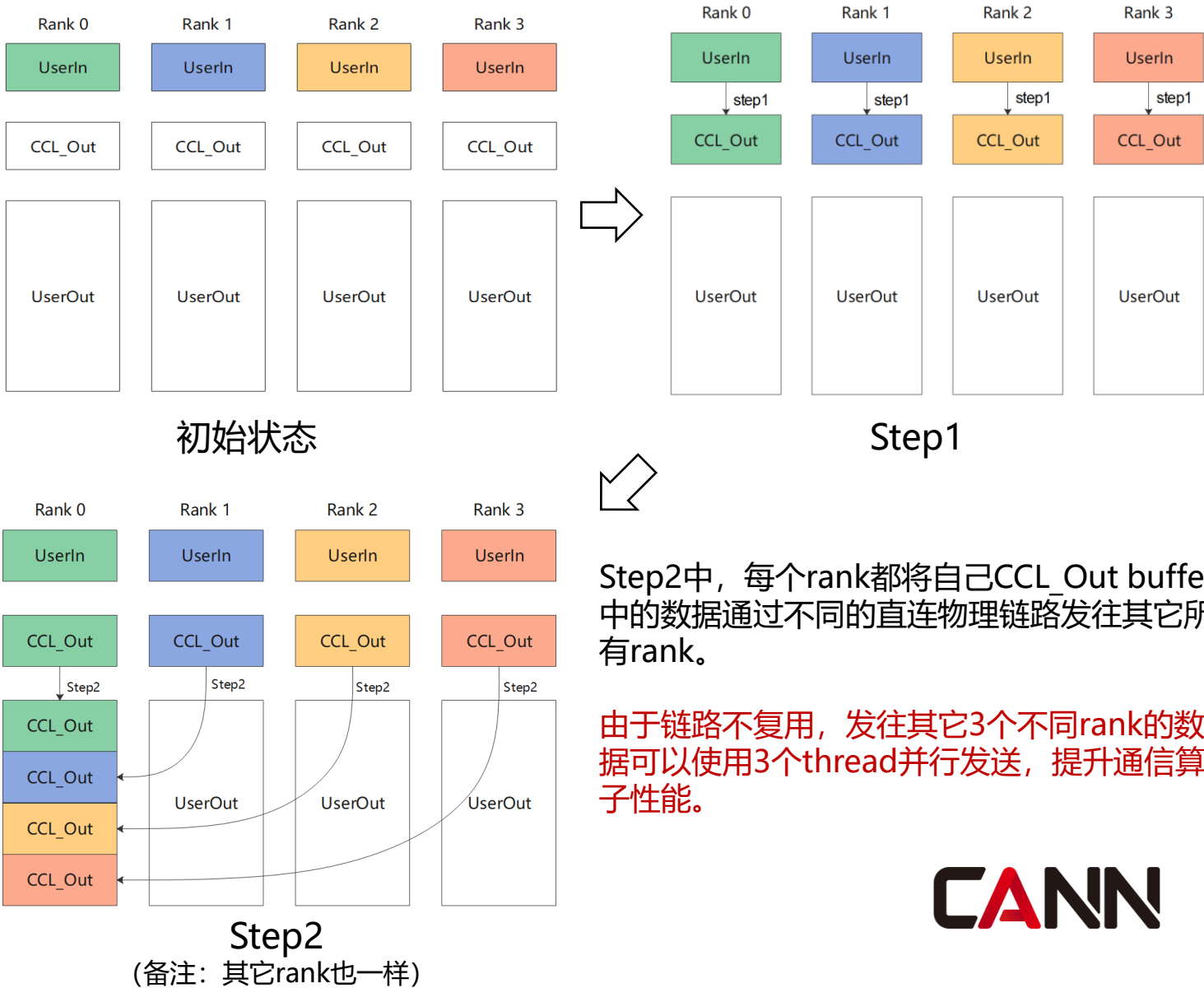
HCCL算子开发介绍 – 更复杂的例子

AllGather算子FullMesh算法



假定通信域中有4个NPU，每个NPU与另外三个NPU都存在物理链路

<https://gitcode.com/cann>



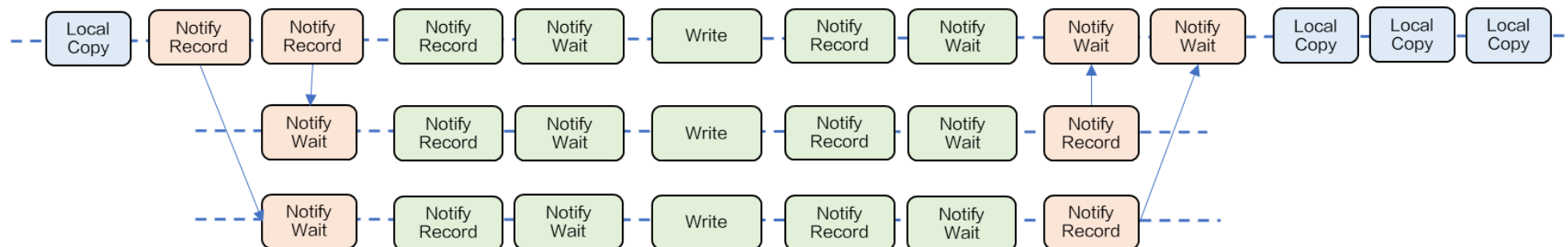
Step2中，每个rank都将自己CCL_Out buffer中的数据通过不同的直连物理链路发往其它所有rank。

由于链路不复用，发往其它3个不同rank的数据可以使用3个thread并行发送，提升通信算子性能。

CANN

HCCL算子开发介绍 – 更复杂的例子

参考链接: https://gitcode.com/cann/hccl/blob/master/examples/05_custom_ops_allgather/aicpu/op_kernel_aicpu/exec_op.cc



```
// 按 ccl buffer 大小, 每 Loop 处理一个 slice
sliceSize = min(maxDataCountPerLoop, 剩余) * dtypeSize;
cclBuffBase = cclMem.addr + sliceSize * myRank; // 本端我这块的位置
for (loop ...) {
    // ① 本地: inputPtr → cclMem[myRank 块]
    HcommLocalCopyOnThread(threads[0], cclBuffBase, inputPtr+off, sliceSize);
    // ② 前同步: 主 thread 通知所有从 thread 开工
    for (i = 1; i < threads.size(); i++)
        HcommThreadNotifyRecordOnThread(threads[0], threads[i], 0);
    for (i = 1; i < threads.size(); i++)
        HcommThreadNotifyWaitOnThread (threads[i], 0, 1836);
    // ③ 数据交换: threads[i] 跟远端 rank i 做 ACK + Write(单边写) + DATA
    for (i = 0; i < threads.size(); i++) {
        ch = channels[i].handle; // 与远端 rank i 的 channel
        remoteCclBuf = channels[i].remoteCclMem.addr
            + sliceSize * myRank; // 写到对端 cclMem 的我这块
        HcommChannelNotifyRecordOnThread(threads[i], ch, NOTIFY_IDX_ACK);
        HcommChannelNotifyWaitOnThread (threads[i], ch, NOTIFY_IDX_ACK, 1836);
        // —— ACK 握手: 双方 staging 就绪
        HcommWriteOnThread(threads[i], ch, remoteCclBuf, cclBuffBase, sliceSize);
        // —— 单边写: 本端 → 远端 cclMem
        HcommChannelNotifyRecordOnThread(threads[i], ch, NOTIFY_IDX_DATA_SIGNAL);
        HcommChannelNotifyWaitOnThread (threads[i], ch, NOTIFY_IDX_DATA_SIGNAL, 1836);
        // —— DATA 握手: 双方写完
    }
    // ④ 后同步: 从 thread 通知主 thread 收齐
    for (i = 1; i < threads.size(); i++)
        HcommThreadNotifyWaitOnThread (threads[i], i, 1836);
    for (i = 1; i < threads.size(); i++)
        HcommThreadNotifyRecordOnThread(threads[i], threads[0], i-1);
    // ⑤ 拷到 output: output[rankId*dataSize+off] ← cclMem[rankId 块]
    for (rankId = 0; rankId < rankSize; rankId++) {
        HcommLocalCopyOnThread(threads[0], outputPtr + rankId*dataSize + off, cclMem.addr + rankId*sliceSize, sliceSize);
    }
}
```

HCCL算子开发最佳实践：模块调用关系

Step1-算法选择：

拓扑解析：通过CANN/hcomm的**通信域拓扑查询API**，感知当前集群拓扑和硬件形态；

算法选择：根据当前**集群拓扑**和**硬件形态**，选择最佳性能的集合通信算法（Mesh、Ring、NHR等）。

Step2-算法资源准备：

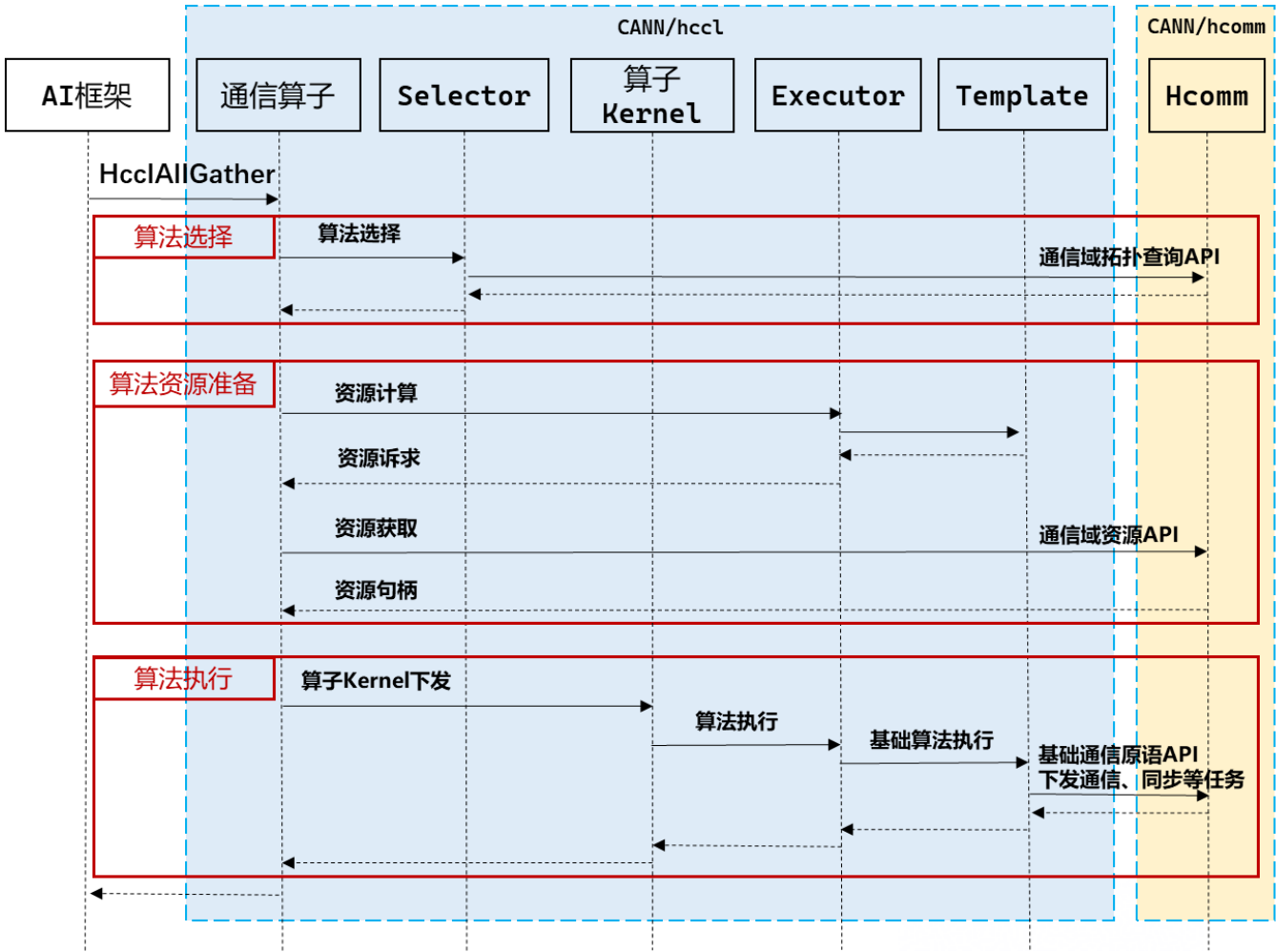
资源计算：依据前一步选择的**算法**以及**拓扑信息**，计算当前算法的通信资源诉求（channel、thread、notify等资源）；

资源获取：根据当前算法资源诉求，通过CANN/hcomm的**通信域资源API**获取对应资源句柄

Step3-算法执行：

Kernel下发：提交当前算子的AI CPU Kernel至任务队列

算法执行：使用创建好的资源，根据当前算法所需步骤，依次调用CANN/hcomm的**基础通信原语API**进行任务下发。



目录

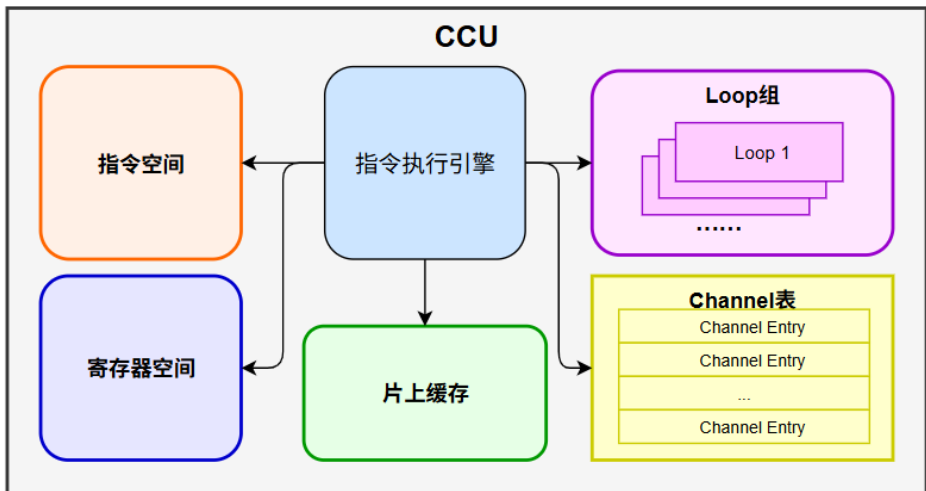
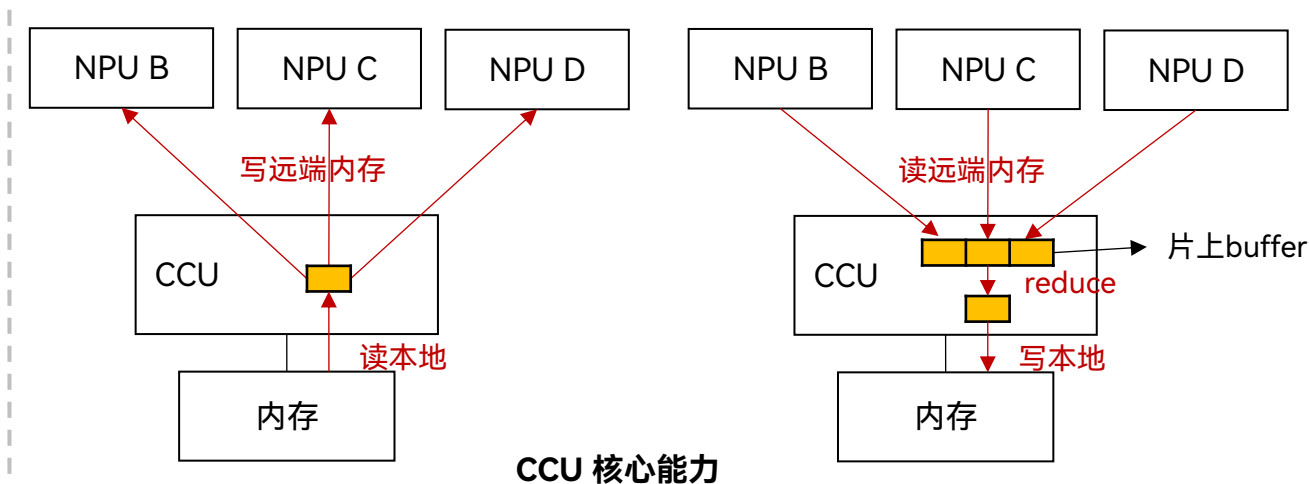
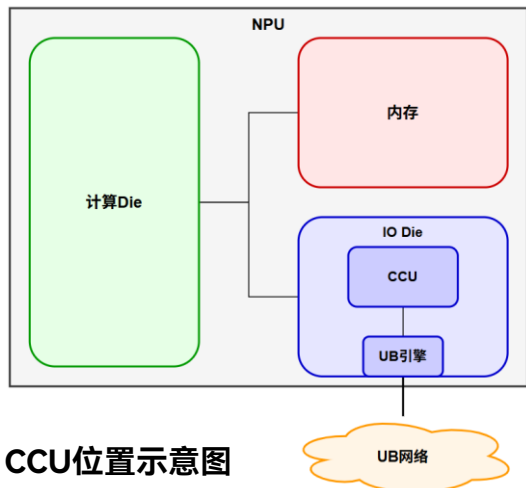
Part 1 HCCL算子层软件架构

Part 2 HCCL算子开发-AICPU+TS引擎

Part 3 HCCL算子开发-CCU引擎

CCU介绍

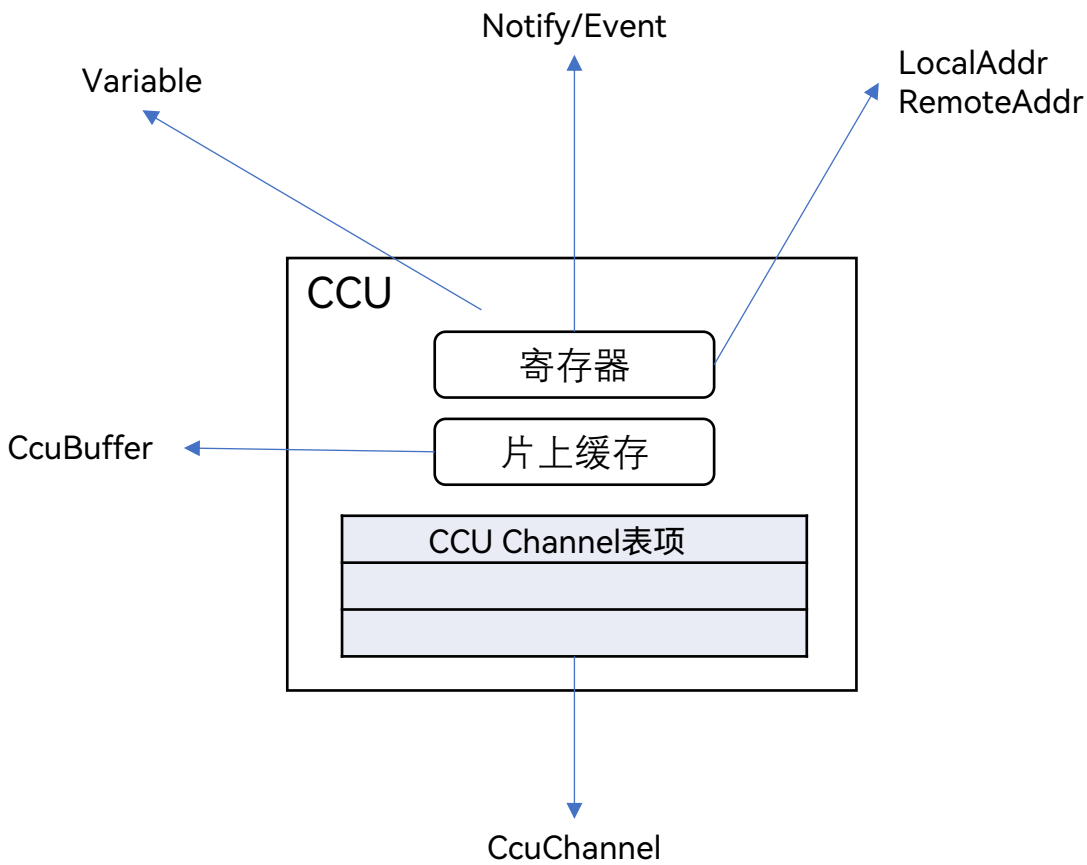
Collective Communication Unit (CCU) 是Ascend 950上新增的用于加速集合通信的专用引擎，CCU根据软件预置的集合通信算法指令，完成通信节点间的同步、地址交换、数据传输以及规约计算，实现集合通信操作。



解决的问题及优势：

- 1、利用CCU片上buffer缓存通信数据，降低访存带宽需求；
- 2、利用片上buffer+内置计算单元执行Reduce操作，确保计算保序；
- 3、专用的通信调度与同步机制，降低时延，不占用计算算力；
- 4、消减用户buffer到CCL Buffer的拷贝

CCU编程模型 - 资源抽象



资源类型	说明	对应资源
ccu::Variable	变量	通用寄存器
ccu::Address	地址	地址寄存器
ccu::Event	事件	同步寄存器
ccu::CcuBuffer	片上缓存分片, 大小为4KB	CCU片上缓存
ccu::LocalAddr	本地地址, 用于通信操作	包含地址和token, 分别由Address和Variable保存
ccu::RemoteAddr	远端地址, 用于通信操作	包含地址和token, 分别由Address和Variable保存

资源创建:

- 单个资源创建: 默认构造函数直接创建
- 批量连续资源创建: 使用ccu::Array创建

注意:

- 1、Loop&LoopGroup对应的CCU并发操作对资源的连续性有要求;
- 2、使用原生数组创建资源 (比如Variable vars[10]) , 不保证资源的连续性。

CCU编程模型 - 计算能力

通用寄存器&地址寄存器

计算能力：赋值和加法

使用立即数赋值

```
1 // 通用寄存器赋值
2 Variable x;
3 x = 5;
4
5 // 地址寄存器赋值
6 uint64_t ptr;
7 ...
8 Address addr;
9 addr = ptr;
```

使用Variable赋值

```
1 Variable x, y;
2 ...
3 Variable v;
4 v = x;
5
6 Address addr;
7 addr = y;
```

加法

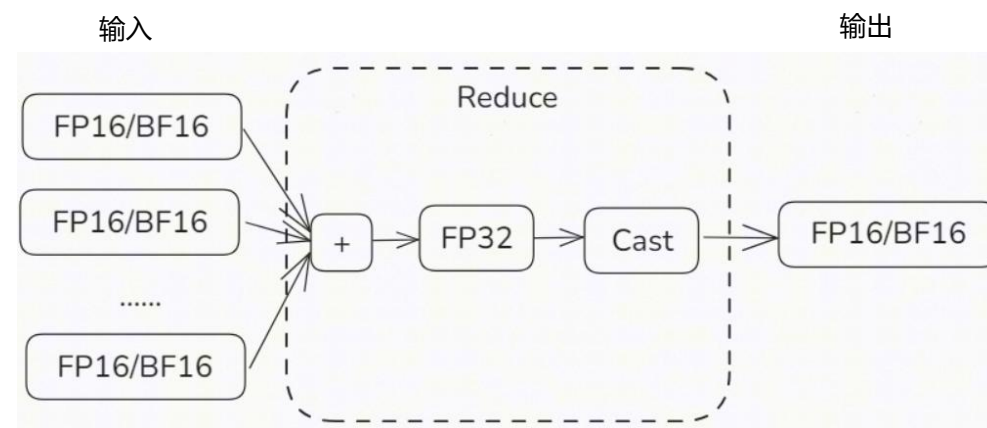
```
1 Variable x=5, y=3, z;
2 z = x+y;
3
4 Address addr1, addr2;
5 ...
6 addr1 += x;
7 addr2 = addr1 + x;
```

CcuBuffer

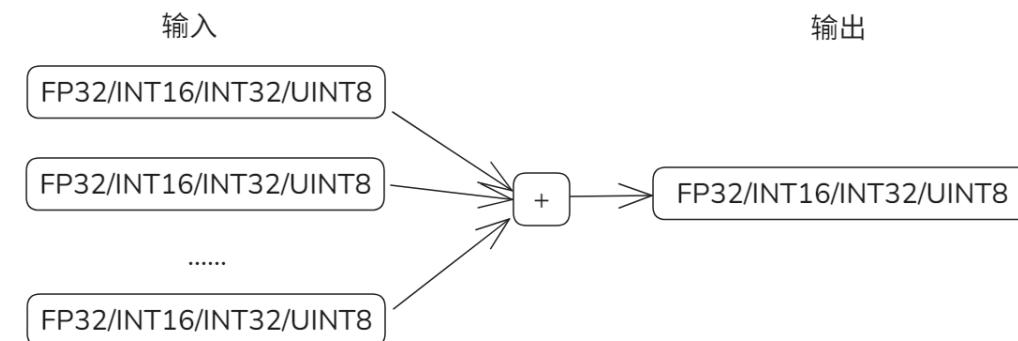
支持最多8个CcuBuffer的按序Reduce计算，支持的Reduce操作：ADD、MAX、MIN。

Reduce操作为ADD：

1) 数据类型为FP16/BF16，内部会提升精度至FP32进行计算

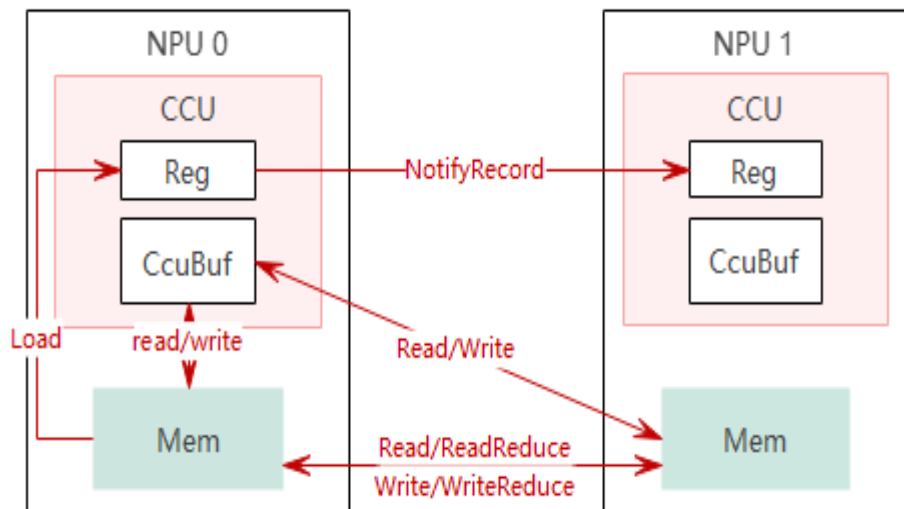


2) 数据类型为FP32/INT16/INT32/UINT8



CCU编程模型 - 数据搬运&通信

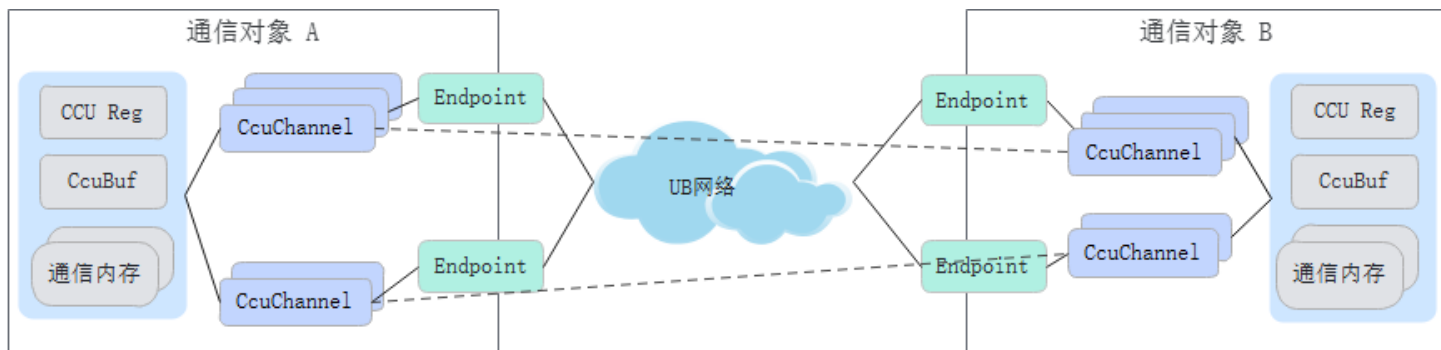
CCU数据搬运能力



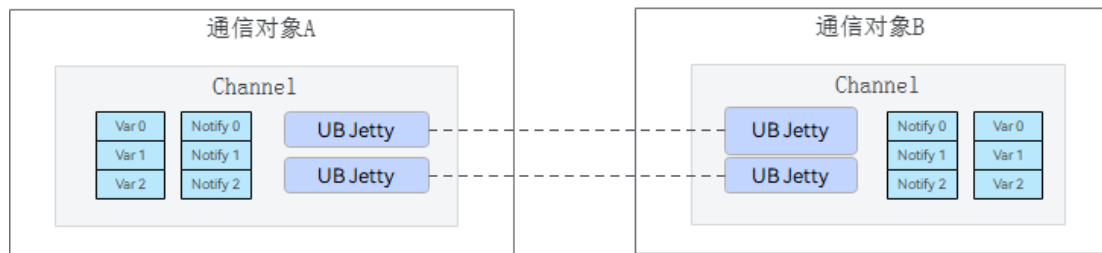
- Local Mem $\leftrightarrow$ Local Variable/Address Reg
- Local Mem $\leftrightarrow$ Local CcuBuf
- Local CcuBuf $\leftrightarrow$ Remote Mem
- Local Mem $\leftrightarrow$ Remote Mem

<https://gitcode.com/cann>

CCU通信模型



- Endpoint：通信对象的数据收发端口；
- CcuChannel：不同通信对象的Endpoint之间建立的通信通道；
- 开发者使用Channel可以读写远端通信对象内存或与远端通信对象进行同步。

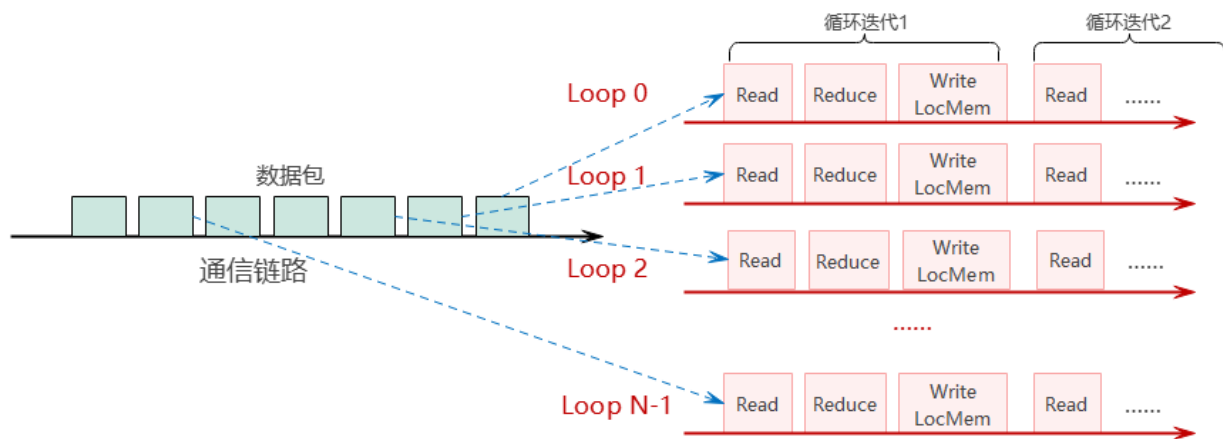


CcuChannel模型

CCU编程模型 - 并发模型

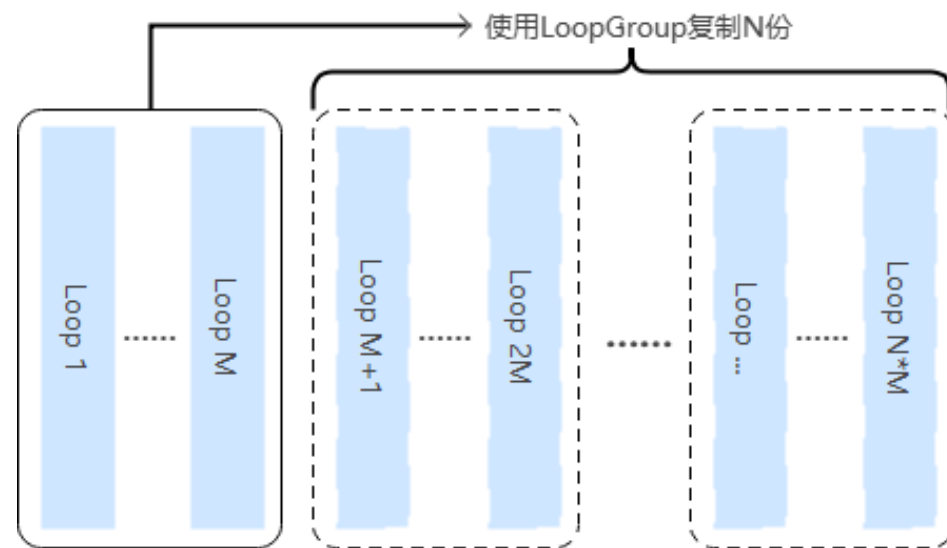
CCU并发模型

并发搬运数据，减少头开销，提升带宽利用率



Loop:

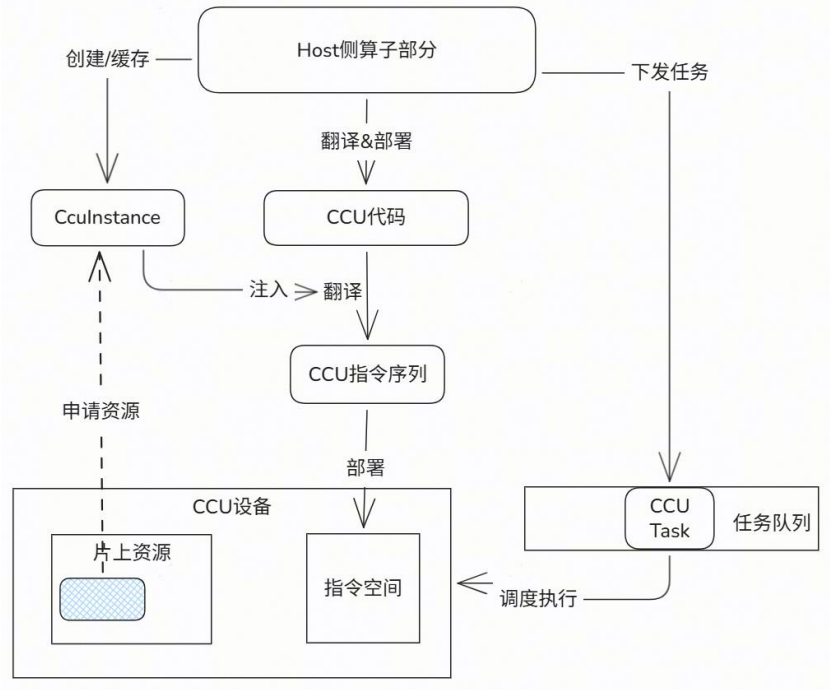
- 一个CCU任务中的基础并发单元;
- Loop内的操作串行, Loop间并行;
- 配合CcuBuf循环搬运数据;
- Loop参数: 循环次数、每次循环操作的内存地址增量偏移;



LoopGroup:

- 将一组Loop复制多份, 增加并发Loop个数;
- 避免重复的指令逻辑手写多份。

CCU编程接口



CCU控制面接口

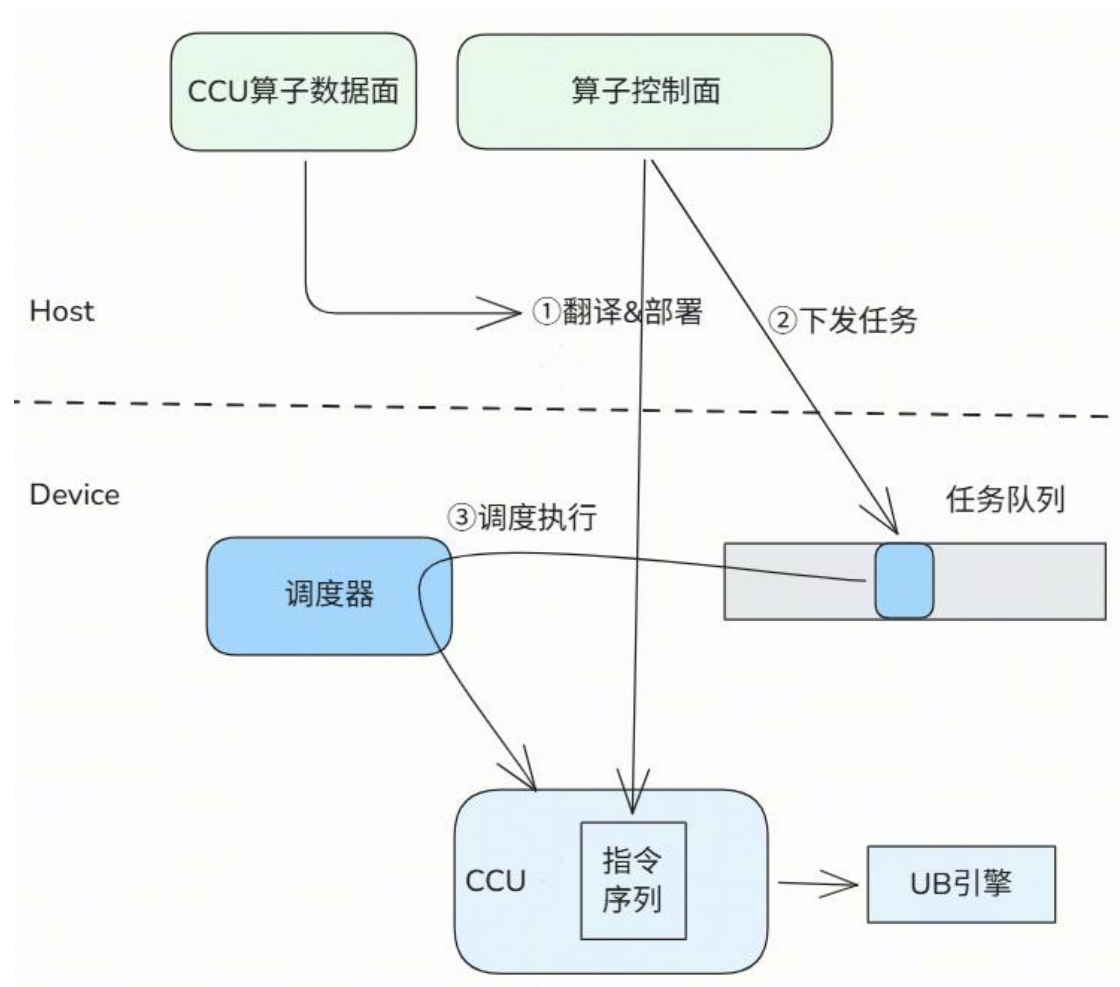
接口	说明
HcommCcuGetMemToken	基于内存地址获取CCU通信需要的token
HcclCommQueryCcuIns	查询通信域中已有的CCU实例，insNum<=1
HcommCcuKernelRegisterStart(ccuInsHandle)	与HcommCcuKernelRegisterEnd配对，Start和End之间可翻译多个可并行执行的CCU Kernel
HcommCcuKernelRegister(ccuInsHandle, diield, kernelFuncName, kernelFunc, kernelArgs, argNum, *kernelHandle)	将CCU Kernel翻译为CCU指令并下发到设备中
HcommCcuKernelRegisterEnd(ccuInsHandle)	与HcommCcuKernelRegisterStart配对，在Start和End之间可翻译多个可并行执行的CCU Kernel

CCU数据面接口

分类	接口	说明
参数加载	-	ccu::LoadArg 加载第argIdx个参数到寄存器
	-	ccu::Load 从内存加载数据到寄存器
	-	ccu::Store 从寄存器保存数据到内存
数据搬运	本地	ccu::LocalCopy LocalAddr↔LocalAddr、LocalAddr↔CcuBuffer、CcuBuffer↔LocalAddr
		ccu::LocalReduce 本地归约 (CcuBuffer到Memory)、多CcuBuffer归约
	远端	ccu::Read 远端→本地 (LocalAddr或CcuBuffer)
		ccu::Write 本地→远端 (LocalAddr或CcuBuffer)
		ccu::ReadReduce 远端读+归约
		ccu::WriteReduce 本地写+归约
同步	本地	ccu::EventRecord 记录事件
		ccu::EventWait 等待事件
	远端	ccu::NotifyRecord 向远端发送通知
		ccu::NotifyWait 等待远端通知
		ccu::WriteVariableWithNotify 写变量并附带通知
流程控制	条件	ccu::CCU_IF 条件开始
		ccu::CCU_ELSE 条件分支
	循环	ccu::CCU_WHILE while循环
	并发	ccu::Loop 执行引擎
		ccu::LoopGroup 循环组
	函数块	ccu::Func 微码函数块
		ccu::CallFunc 调用函数块



HCCL算子执行流程：CCU通信引擎



算子控制面(算子入口)

- 执行算法选择
- 申请&管理资源
- 翻译CCU算子数据面实现
- 下发部署翻译后的CCU指令流
- 下发CCU任务

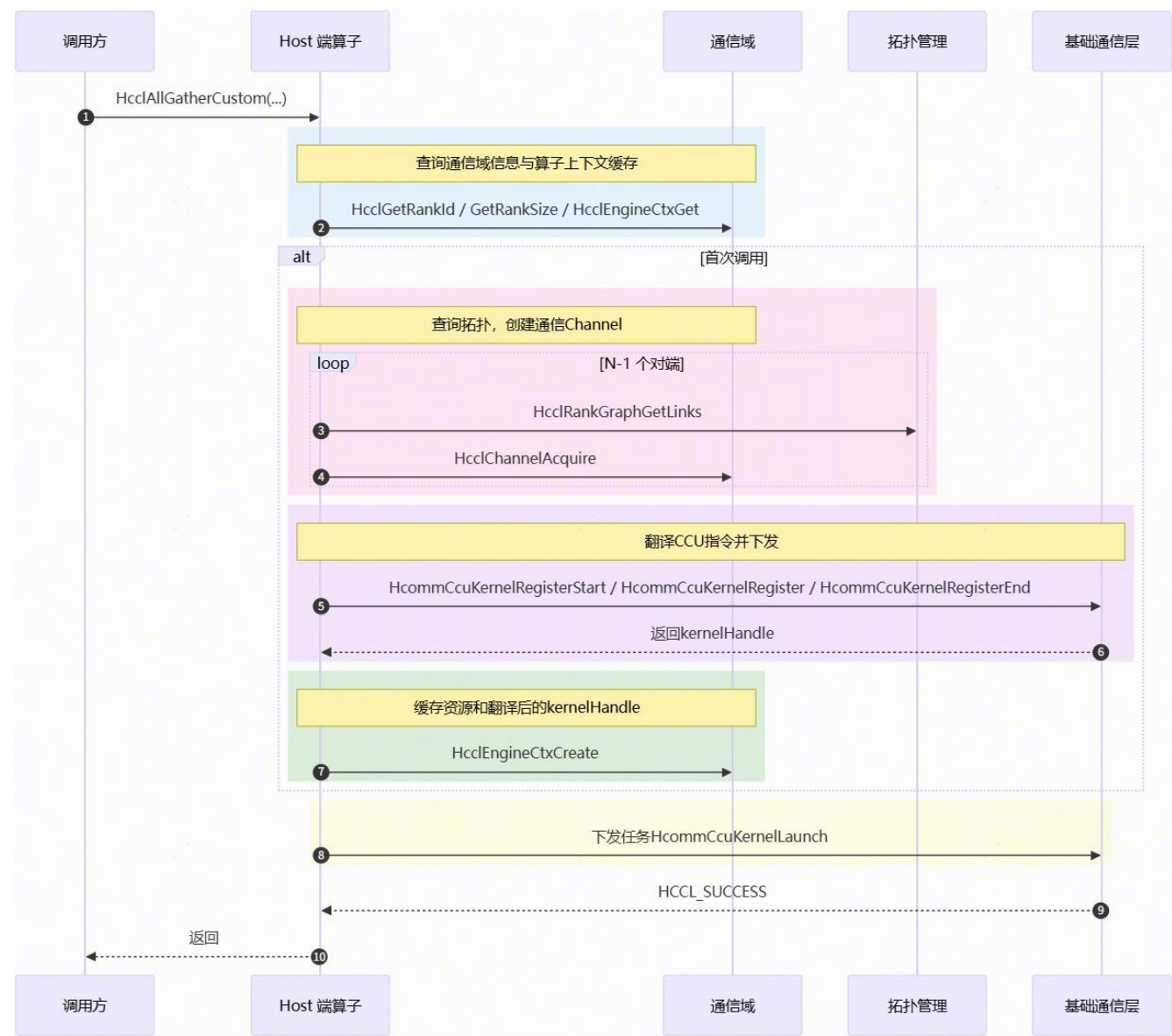
算子数据面(CCUCPU Kernel) :

- 基于CCU数据面编程接口，实现通信算子的数据搬运逻辑
- 数据面程序需要翻译为CCU指令流并部署到CCU设备上
- Host下发到任务队列的任务被调度器调度至CCU执行
- CCU执行对应的指令流，并利用UB引擎完成数据搬运

HCCL算子开发介绍：CCU AllGather算子样例

参考链接：https://gitcode.com/cann/hccl/blob/master/examples/05_custom_ops_allgather/ccu/op_host/allgather.cc

算子Host侧流程介绍



```
HcclResult HcclAllGatherCustom(void *sendBuf, void *recvBuf, uint64_t sendCount,
                               HcclDataType dataType, HcclComm comm, aclrtStream stream)
{
    // ===== 1. 获取通信域信息 =====
    HcclGetRankId(comm, myRank);
    HcclGetRankSize(comm, rankSize);
    // ===== 2. 资源缓存检查 (Engine Ctx) =====
    if (HcclEngineCtxGet(comm, tag, COMM_ENGINE_CCU, &ctx, &size) == SUCCESS) {
        resCtxHost.DeSerialize(ctxData);
    } else {
        // 2b. 申请 thread (封装主流, 创建通知)
        HcclThreadAcquireWithStream(comm, COMM_ENGINE_CCU, stream, notifyNum, &thread);
        resCtxHost.threads.push_back(thread);
        // 2c. 对每个远端 rank: 查询拓扑 + 申请 CCU 通道
        for (remoteRank = 0..rankSize) {
            if (remoteRank == myRank) continue;
            HcclRankGraphGetLinks(comm, layer, myRank, remoteRank, &linkList, &listSize);
            HcclChannelDescInit(&desc, 1);
            desc.remoteRank = remoteRank;
            ...
            HcclChannelAcquire(comm, COMM_ENGINE_CCU, &desc, 1, &channelHandle);
            kernelChannels.push_back(channelHandle);
        }
        // 2d. 查询 CCU 实例
        HcclCommQueryCcuIns(comm, &insHandle, &insNum);
        HcommCcuKernelRegisterStart(insHandle);
        HcommCcuKernelRegister(insHandle,
                               "CcuAllGatherMesh1DMem2MemKernel", // kernel 名称
                               (void*)CcuAllGatherMesh1DMem2MemKernel, // kernel 函数指针
                               kernelArg, // 含 channels 信息的参数
                               &kernelHandle); // 输出 kernel 句柄
        HcommCcuKernelRegisterEnd(insHandle);
        resCtxHost.ccuKernels[0] = kernelHandle;
        // 2e. 序列化资源上下文, 写入 EngineCtx (下次复用)
        seq = resCtxHost.Serialize();
        HcclEngineCtxCreate(comm, tag, COMM_ENGINE_CCU, seq.size(), &ctx);
        memcpy(ctx, seq.data(), seq.size());
    }
    // ===== 3. 下发 CCU 任务 =====
    // 获取内存 token (CCU 地址转换)
    HcommCcuGetMemToken(sendBuf, dataSize, &token);
    for (loop = 0; loop < loopCount; loop++) {
        // 计算分片地址 / 大小
        taskArgs = {inputAddr, outputAddr, token,
                    sliceInputOffset, sliceOutputOffset, sliceSize, ...};
        HcommCcuKernelLaunch(resCtx.threads[0], resCtx.ccuKernels[0],
                             taskArgs.data(), taskArgs.size());
    }
    return HCCL_SUCCESS;
}
```

HCCL算子开发介绍：CCU AllGather算子样例

参考链接：https://gitcode.com/cann/hccl/blob/master/examples/05_custom_ops_allgather/ccu/op_kernel_ccu/ccu_kernel.cc

算子Device侧流程介绍

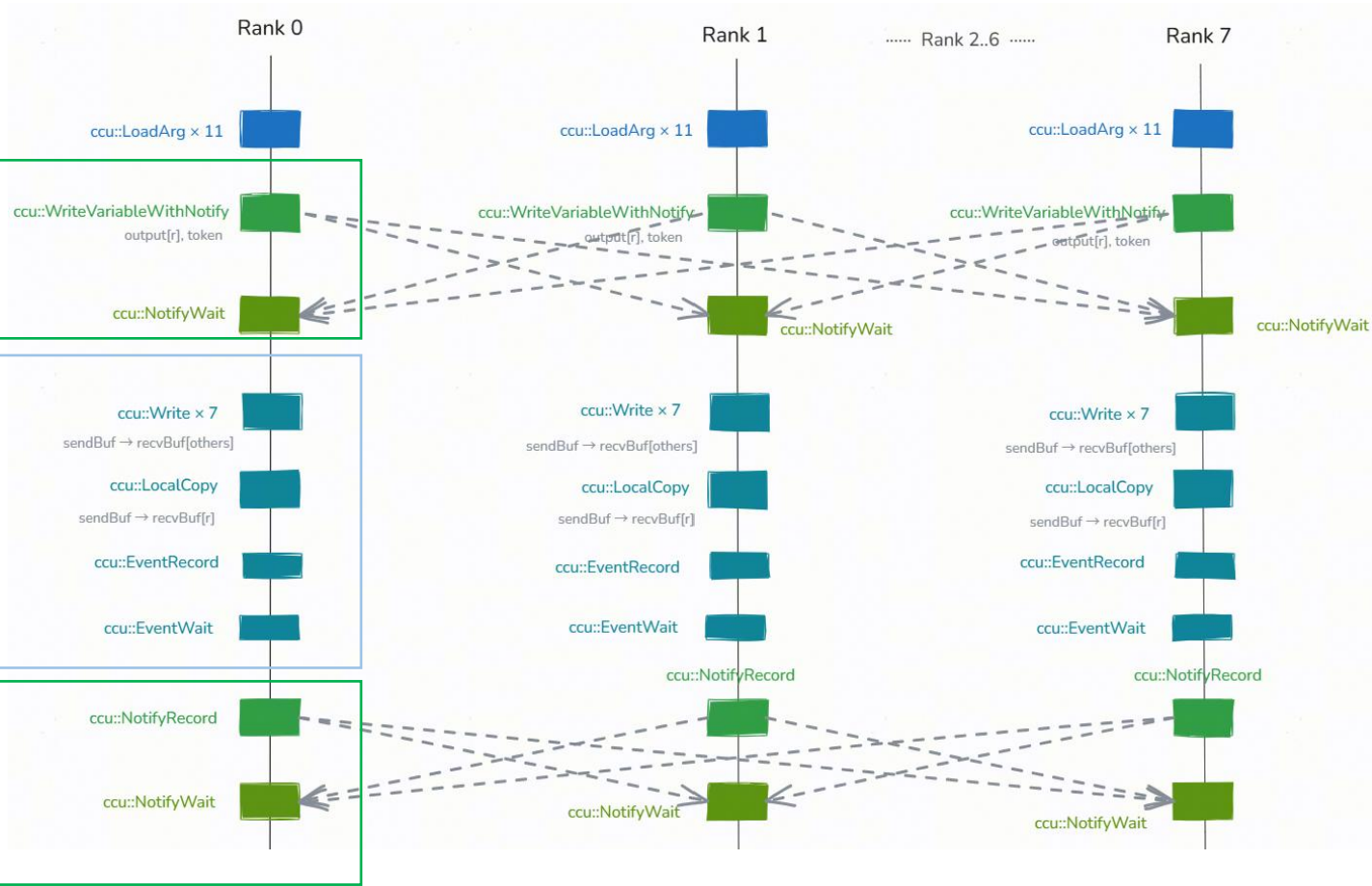
```
CcuResult CcuAllGatherMesh1DMem2MemKernel(CcuKernelArg arg)
{
    // ... 省略 ① 资源初始化 / ② 加载参数

    // ③ 前同步: 用 WriteVariableWithNotify 一次性写变量+触发 notify
    for (auto ch : channels) {
        ccu::WriteVariableWithNotify(ch, myOutput, OUTPUT_XN_ID, CKE_IDX_0, ...);
        ccu::WriteVariableWithNotify(ch, myToken, TOKEN_XN_ID, CKE_IDX_0, ...);
        ccu::NotifyWait(ch, CKE_IDX_0, OUTPUT_BIT | TOKEN_BIT);
    }

    // ④ 算法: 本地 + 跨卡并发, 共享 event 同步
    for (auto peer : ranks) {
        if (peer != myRank) {
            ccu::Write(channels[peer], remoteDst[peer], sendBuf,
                       sliceSize, event, 1 << myRank); // 跨卡写入
        } else {
            ccu::EventRecord(event, 1 << myRank);
            ccu::LocalCopy(...); // 本卡: sendBuf -> recvBuf[myRank]
        }
    }
    ccu::EventWait(event, ALL_RANKS_MASK); // 本卡任务全部发起

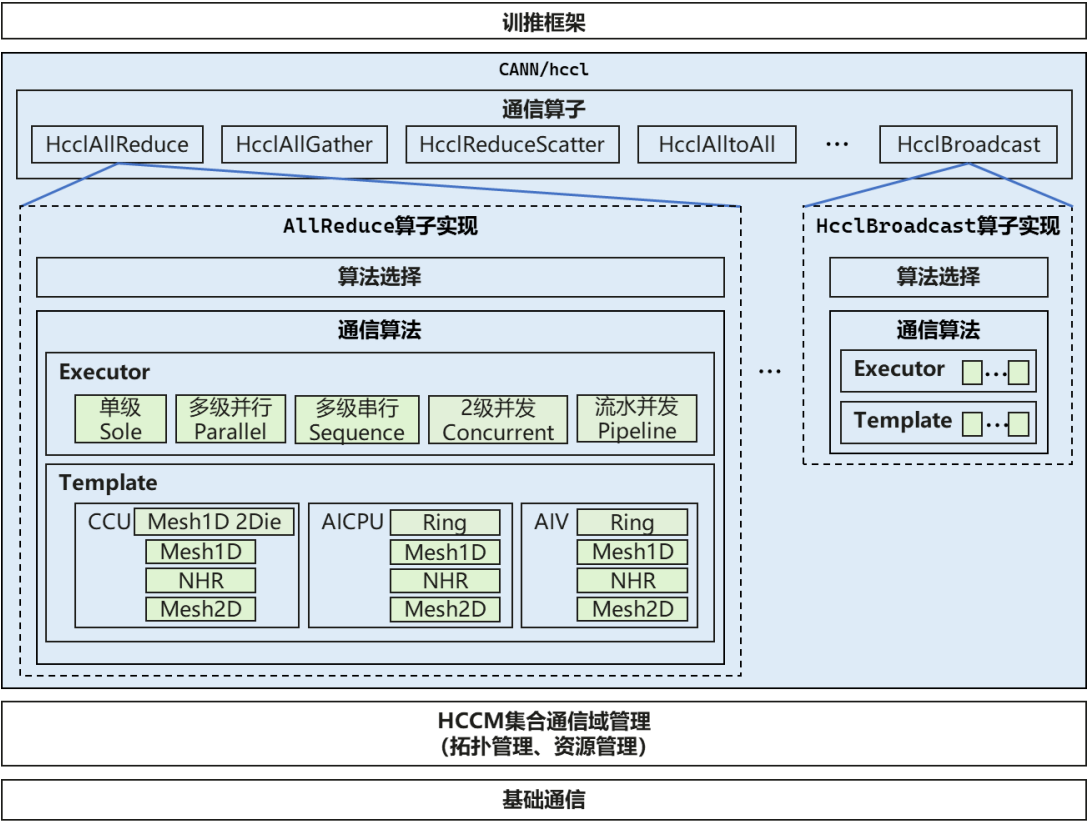
    // ⑤ 后同步: Record + Wait 等待对端完成
    for (auto ch : channels) {
        ccu::NotifyRecord(ch, CKE_IDX_0, POST_SYNC_BIT);
    }
    for (auto ch : channels) {
        ccu::NotifyWait(ch, CKE_IDX_0, POST_SYNC_BIT);
    }

    return CCU_SUCCESS;
}
```

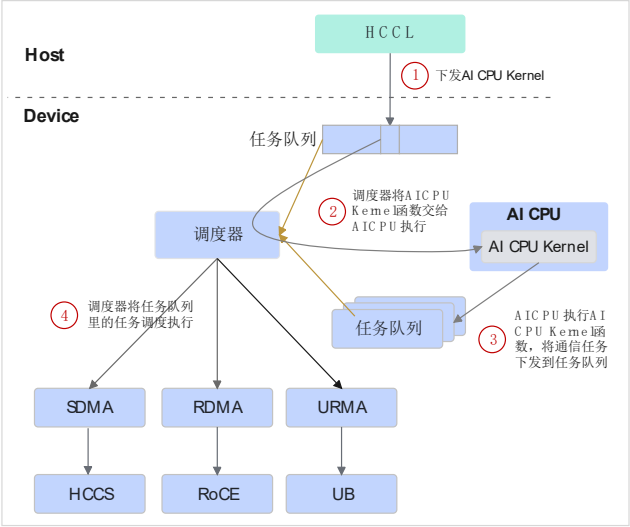


课程小结

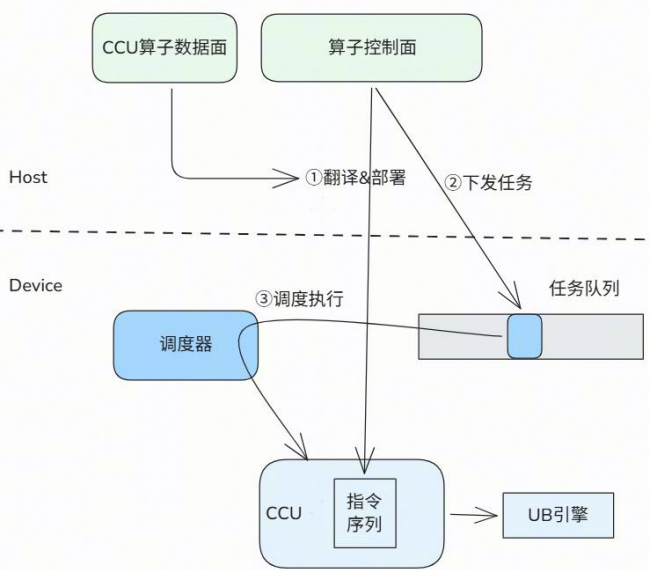
- HCCL算子层软件架构



- AICPU+TS引擎的通信算子开发



- CCU引擎的通信算子开发



Thank you.



HCCL开源社区技术交流群

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯